

Software Requirements Specification

Restaurant App MVP — Frontend Only, React Native Assignment No. 1 · Fall 2026

Document purpose. This SRS defines the client-side requirements for a restaurant mobile MVP. It is intentionally scoped to mock/local data and frontend behaviour; no backend service is required.

1. Introduction

1.1 Purpose. The product lets a diner browse a restaurant menu, search and sort dishes, manage a cart, apply promotions, reserve a table, place an order and track its progress. A manager can review incoming orders, reservations and menu availability. The MVP is designed as an academic React Native prototype.

1.2 Scope. In scope: login/signup validation; role-based customer/manager access; menu categories and daily specials; search with debounce; favourites; cart quantity and notes; promo codes; order summary; Dine-in/Takeaway selection; table reservation; order tracking; manager dashboard; light/dark theme; local persistence with AsyncStorage; responsive mobile UI; required React hooks and custom hooks.

Out of scope. Real payment processing, production authentication, backend APIs, server database, push notifications, real-time multi-device synchronization, external food-delivery integration and production security.

1.3 Definitions. SRS = Software Requirements Specification; MVP = Minimum Viable Product; UML = Unified Modeling Language; Hook = React function for state/lifecycle behaviour; Context = React mechanism for shared state; Reducer = pure function that calculates next state from state + action; Mock Data = local data used instead of a backend; FlatList = React Native virtualized list component; AsyncStorage = local key-value persistence; Memoization = caching derived values/references to reduce unnecessary work.

2. Overall Description

2.1 Product perspective. The app is a standalone Expo/React Native frontend. Data starts from local JavaScript datasets and runtime state is held in Context, reducers and screen state. AsyncStorage persists orders, reservations and manager menu edits on the device.

Role	Goals / permissions
Customer	Authenticate, browse/search/sort menu, favourite items, add/update cart, apply promo, review totals, reserve/cancel tables, choose Dine-in or Takeaway, place and track orders, change theme, logout.
Restaurant Manager	Authenticate as manager, view/update incoming orders, accept/decline reservations, manage menu availability, edit menu pricing, and access manager-only dashboard features.

2.2 Self-interview / client meeting. The customer persona says: "I am hungry and in a hurry; I want daily specials, a quick table booking and food ready when I arrive." From that interview, the MVP prioritizes fast menu discovery, visible availability, simple reservation rules, a short cart-to-order flow and live demo tracking.

2.3 User Stories

US-01. As a Customer, I want to see daily specials so that I can decide quickly what to eat.

US-02. As a Customer, I want to search dishes by name or description so that I do not have to scroll through the full menu.

US-03. As a Customer, I want to filter by category so that I can focus on starters, mains, desserts or drinks.

US-04. As a Customer, I want to reserve a table for a chosen time so that I can arrive with seating arranged.

US-05. As a Customer, I want unavailable time slots disabled so that I do not submit an impossible reservation.

US-06. As a Customer, I want to add special instructions so that the restaurant receives notes such as "no onions".

US-07. As a Customer, I want to apply a promo code so that the order total reflects my available discount.

US-08. As a Customer, I want to choose Dine-in or Takeaway so that the order contains the correct fulfilment information.

US-09. As a Restaurant Manager, I want to update order status so that customers can see progress.

US-10. As a Restaurant Manager, I want to toggle menu availability so that the customer menu reflects what the kitchen can currently serve.

3. Functional Requirements

Authentication

FR-01. The system shall validate login email format.

FR-02. The system shall require a password of at least eight characters containing at least one digit.

FR-03. The system shall require matching password and confirm-password fields during signup.

FR-04. The system shall authenticate against the local users dataset.

FR-05. The system shall show a one-second submitting state and disable the submit button while authentication is simulated.

FR-06. The system shall route customer and manager users to their role-appropriate application area.

Menu

FR-07. The system shall load at least fifteen menu items across Starters, Main, Desserts and Drinks from local data.

FR-08. The system shall show item name, description, price, availability and Daily Special state.

FR-09. The system shall grey out unavailable items and disable their Add button.

FR-10. The system shall display category chips and update visible items when the selected category changes.

FR-11. The system shall support pull-to-refresh of the local menu dataset.

FR-12. The system shall display the number of currently visible menu items.

Search

FR-13. The system shall focus the search input through a ref when the search control is used.

FR-14. The system shall debounce search input by 400 milliseconds.

FR-15. The system shall provide a clear-search action that empties the query and retains focus.

FR-16. The system shall retain the last five search terms as suggestions.

FR-17. The system shall prevent consecutive duplicate search terms from being added.

FR-18. The system shall show a friendly empty state when no menu item matches.

Cart

FR-19. The system shall add an available menu item to the cart and increase quantity when the same item is added again.

FR-20. The system shall increment and decrement quantities without mutating previous reducer state.

FR-21. The system shall remove an item when its quantity reaches zero.

FR-22. The system shall allow special instructions per cart item.

FR-23. The system shall accept WELCOME10 and FEAST20 promo codes and reject invalid codes.

FR-24. The system shall calculate and display a live subtotal, service charge, tax, discount and grand total.

Reservation

FR-25. The system shall offer hourly reservation slots from 12:00 through 22:00.

FR-26. The system shall disable a slot when no table with enough seats is free.

FR-27. The system shall validate party size between 1 and 12.

FR-28. The system shall validate Pakistani mobile format 03XX-XXXXXXX.

FR-29. The system shall reject past dates and bookings less than one hour ahead.

FR-30. The system shall show a confirmation before saving a reservation.

FR-31. The system shall allow a customer to cancel a saved reservation after confirmation.

Orders

FR-32. The system shall create an order with id, items, total, type, status and timestamp.

FR-33. The system shall allow Dine-in with a table number or Takeaway with a pickup time.

FR-34. The system shall progress demo orders from Pending to Preparing after 10 seconds, Ready after 20 seconds and Served after 30 seconds.

FR-35. The system shall show order progress and elapsed time and clear timer resources on unmount.

FR-36. The system shall persist orders locally and restore them when the app starts.

FR-37. The system shall allow the manager to update an order status manually.

Manager Dashboard

FR-38. The system shall show the manager dashboard only for a user whose role is manager.

FR-39. The system shall provide Incoming Orders, Reservations and Menu Management areas.

FR-40. The system shall allow the manager to accept or decline pending reservations.

FR-41. The system shall allow the manager to toggle menu availability and edit prices.

FR-42. The system shall persist manager menu edits and make them visible to the customer menu.

4. Non-Functional Requirements

Usability: Controls shall use clear labels, visible validation and disabled states. Primary actions shall be reachable on common phone widths.

Performance: Menu data is local; FlatList shall be used for the menu; search shall debounce at 400 ms; expensive derived menu/order totals shall use useMemo where appropriate; MenuItemCard shall use React.memo with stable callbacks.

Responsiveness: Layouts shall use flexible widths, SafeAreaView and scrollable content so common Android/iOS phone sizes can display the full flow.

Maintainability: Data, reducers, hooks, contexts, components, screens and navigation contracts shall be separated under src. Reducers must be pure and hooks must be called at the top level.

Data handling: No backend is required. Users, categories, menu items, tables, reservations, orders and promo codes are local datasets; orders/reservations/menu edits are persisted through AsyncStorage.

Reliability: Loading timers and order timers shall be cleared in useEffect cleanup functions; invalid actions shall not mutate previous state.

5. Client-Side Data Model

Dataset	Representative fields	Purpose
Users	id, name, email, password, role	Mock authentication and role routing.
Categories	name	Menu category chips.
MenuItems	id, name, description, price, category, image, isSpecial, isAvailable	Customer menu and manager menu management.
Tables	id, seats	Reservation capacity/availability.
Reservations	id, date, time, partySize, phone, tableId, status	Local table bookings.
Orders	id, items, total, type, table/pickupTime, status, timestamp	Order creation, tracking and manager updates.
PromoCodes	code, discountPercent	Local discount validation.

6. UML Diagrams

Use Case Diagram. Shows Customer and Restaurant Manager actors and the major application use cases, including ordering, reservation and manager operations. See the corresponding PNG in [A1/UML/](#).

Class Diagram. Models User, Customer, Manager, Category, MenuItem, Cart, CartItem, Order, Reservation and Table with their relationships and multiplicities. See the corresponding PNG in [A1/UML/](#).

Sequence Diagram. Traces the customer order path from Add to Cart through review/place order to the Order Tracking screen. See the corresponding PNG in [A1/UML/](#).

State Machine Diagram. Models the Order lifecycle: Pending → Preparing → Ready → Served, with Cancelled as a terminal alternative. See the corresponding PNG in [A1/UML/](#).

Component Diagram. Shows the React Native application layers: screens/components, Context providers, reducers, custom hooks and local data/persistence. See the corresponding PNG in [A1/UML/](#).

7. MVP Frontend Development

7.1 Login and Signup

Purpose: authenticate a mock customer/manager. UI: controlled inputs, role selector, validation messages, password toggle, loading indicator. Entry: unauthenticated app. Exit: customer menu or manager dashboard. Hooks: useState, useForm/useContext integration.

7.2 Menu Browsing

Purpose: discover dishes. UI: categories, cards, specials, availability, favourites, add-to-cart. Data: src/data/menu.js and categories.js. Hooks: useState, useEffect, useCallback, useMemo.

7.3 Search and Scroll Controls

Purpose: fast menu discovery. UI: search input, recent suggestions, clear/focus, Back to Top, render counter. Hooks: useRef and useDebounce plus useMemo.

7.4 Profile and Theme

Purpose: show account details and change theme. UI: name, email, role, theme switch and logout. Hooks: Context consumers.

7.5 Cart

Purpose: edit cart state. UI: quantity stepper, remove, notes, promo, fulfilment type. Hook: useReducer through CartContext.

7.6 Order Summary

Purpose: review all financial components before submission. Constants: service 5%, sales tax 15%. Hooks: useMemo for derived totals.

7.7 Table Reservation

Purpose: reserve and cancel tables. UI: date, party size, phone, hourly slots, confirmation and My Reservations. Hook: useReservation.

7.8 Order Tracking and Manager Dashboard

Purpose: combine the full workflow. Customer sees timed status progress; manager sees orders/reservations/menu management. Persistence uses AsyncStorage and timers are cleaned up.

8. Hook Usage Matrix

Hook / API	Where used	Reason
useState	Login, Menu, Cart, Reservation, Manager UI	Local UI state and form values.

useEffect	Menu loading, order timers, AsyncStorage	Lifecycle, simulated async work and persistence.
useRef	Search input, FlatList, render counter, previous query	Persistent mutable values without re-render.
useContext	Auth, Theme, Cart, Orders contexts	Shared state without prop drilling.
useReducer	Cart and Orders	Pure multi-action state transitions.
useMemo	Menu filtering/sorting and order totals	Derived calculations and stable context values.
useCallback	Add/favourite handlers, theme/auth actions	Stable function references for memoized children.
React.memo	MenuItemCard	Avoid unnecessary card re-renders.
Custom hooks	useForm, useDebounce, useReservation	Reusable logic extracted from screens.

9. Submission Structure

A1/SRS.pdf — this document.

A1/UML/ — five UML PNG diagrams.

src/data/ — mock datasets.

src/context/ — Auth, Theme, Cart and Orders contexts.

src/reducers/ — cart and orders reducers plus cart test cases.

src/hooks/ — useForm, useDebounce and useReservation.

src/screens/ — screen contracts/reusable Order Summary implementation.

src/navigation/ — route contract for customer and manager flows.

README.md — installation, credentials, hooks matrix, testing and scope.